

SESSION 3 — Selecting & Filtering Data in Pandas

In real-world data analysis, we rarely use the entire dataset.

Most of the time we need to:

- Select specific columns
- Select specific rows
- Filter rows based on conditions
- Query important information from the dataset

Pandas provides powerful tools for selecting and filtering data.

The two most important methods are:

- loc → label based selection
- iloc → position based selection

▼ Step 1 — Create a Sample Dataset

We will create a small dataset of customers.

Columns include:

- Customer Name
- City
- Revenue

```
import pandas as pd

data = {
    "Customer": ["Aman", "Priya", "Rahul", "Sneha", "Karan"],
    "City": ["Delhi", "Mumbai", "Delhi", "Bangalore", "Mumbai"],
    "Revenue": [12000, 8000, 15000, 7000, 20000]
}

df = pd.DataFrame(data)

df
```

	Customer	City	Revenue
0	Aman	Delhi	12000
1	Priya	Mumbai	8000
2	Rahul	Delhi	15000
3	Sneha	Bangalore	7000
4	Karan	Mumbai	20000

▼ Selecting Columns

We can select columns using the column name.

Syntax:

```
df["column_name"]
```

```
df["Revenue"]
```

```
Revenue
0    12000
1     8000
2    15000
3     7000
4    20000

dtype: int64
```

✓ Selecting Multiple Columns

Syntax:

```
df[["column1", "column2"]]
```

```
df[["Customer", "Revenue"]]
```

	Customer	Revenue
0	Aman	12000
1	Priya	8000
2	Rahul	15000
3	Sneha	7000
4	Karan	20000

✓ Selecting Rows using loc

loc selects rows based on **labels or index values**.

Syntax:

```
df.loc[row_index]
```

```
df.loc[row_index, column_name]
```

```
# Select first row
```

```
df.loc[0]
```

	0
Customer	Aman
City	Delhi
Revenue	12000

dtype: object

```
# Select specific value
```

```
df.loc[2, "Revenue"]
```

```
np.int64(15000)
```

✓ Selecting Rows using iloc

iloc selects rows using **numerical positions**.

Syntax:

```
df.iloc[row_position]
```

```
df.iloc[row_position, column_position]
```

```
# Select third row

df.iloc[2]
```

2	
Customer	Rahul
City	Delhi
Revenue	15000

dtype: object

```
# Select specific value

df.iloc[2, 2]

np.int64(15000)
```

Boolean Filtering

Filtering means selecting rows based on conditions.

Example:

Find customers whose revenue is greater than 10000.

```
df["Revenue"] > 10000
```

Revenue	
0	True
1	False
2	True
3	False
4	True

dtype: bool

```
df[df["Revenue"] > 10000]
```

	Customer	City	Revenue
0	Aman	Delhi	12000
2	Rahul	Delhi	15000
4	Karan	Mumbai	20000

```
# Customers with revenue > 10000 and city = Delhi

(df["Revenue"] > 10000) & (df["City"] == "Delhi")
```

0	
0	True
1	False
2	True
3	False
4	False

dtype: bool

```
# Customers with revenue > 10000 and city = Delhi

df[(df["Revenue"] > 10000) & (df["City"] == "Delhi")]
```

	Customer	City	Revenue
0	Aman	Delhi	12000
2	Rahul	Delhi	15000

Querying Data

Pandas also provides the `query()` method.

Syntax:

```
df.query("condition")
```

```
df.query("Revenue > 10000")
```

	Customer	City	Revenue
0	Aman	Delhi	12000
2	Rahul	Delhi	15000
4	Karan	Mumbai	20000

Example Case Study

Select customers whose revenue is greater than 10000.

```
high_revenue_customers = df[df["Revenue"] > 10000]
```

```
high_revenue_customers
```

	Customer	City	Revenue
0	Aman	Delhi	12000
2	Rahul	Delhi	15000
4	Karan	Mumbai	20000

SESSION 4 — Sorting & Reindexing

Sorting is an important operation when analyzing data.

Examples:

- Sorting products by price
- Ranking students by marks
- Sorting customers by revenue

Pandas provides functions for sorting rows and indices.

Sorting Values

Function:

```
sort_values()
```

Syntax:

```
df.sort_values(by="column_name")
```

```
df.sort_values(by="Revenue")
```

	Customer	City	Revenue
3	Sneha	Bangalore	7000
1	Priya	Mumbai	8000
0	Aman	Delhi	12000
2	Rahul	Delhi	15000
4	Karan	Mumbai	20000

✎ Sorting in Descending Order

We can sort values in descending order using:

`ascending=False`

```
df.sort_values(by="Revenue", ascending=False)
```

	Customer	City	Revenue
4	Karan	Mumbai	20000
2	Rahul	Delhi	15000
0	Aman	Delhi	12000
1	Priya	Mumbai	8000
3	Sneha	Bangalore	7000

✎ Sorting Multiple Columns

Syntax:

```
df.sort_values(by=["column1","column2"])
```

```
df.sort_values(by=["City", "Revenue"])
```

	Customer	City	Revenue
3	Sneha	Bangalore	7000
0	Aman	Delhi	12000
2	Rahul	Delhi	15000
1	Priya	Mumbai	8000
4	Karan	Mumbai	20000

✎ Sorting by Index

We can also sort rows based on index values.

Function:

```
sort_index()
```

```
df.sort_index()
```

	Customer	City	Revenue
0	Aman	Delhi	12000
1	Priya	Mumbai	8000
2	Rahul	Delhi	15000
3	Sneha	Bangalore	7000
4	Karan	Mumbai	20000

✎ Resetting Index

After filtering or sorting, index numbers may become irregular.

We can reset them using:

`reset_index()`

```
sorted_df = df.sort_values(by="Revenue")
```

```
sorted_df.reset_index()
```

	index	Customer	City	Revenue
0	3	Sneha	Bangalore	7000
1	1	Priya	Mumbai	8000
2	0	Aman	Delhi	12000
3	2	Rahul	Delhi	15000
4	4	Karan	Mumbai	20000

✓ Removing Old Index

We can drop the old index using:

`reset_index(drop=True)`

```
sorted_df.reset_index(drop=True)
```

	Customer	City	Revenue
0	Sneha	Bangalore	7000
1	Priya	Mumbai	8000
2	Aman	Delhi	12000
3	Rahul	Delhi	15000
4	Karan	Mumbai	20000

✓ Reindexing Data

Reindexing allows us to reorder or redefine the index.

Syntax:

`df.reindex(new_index)`

```
df.reindex([4,3,2,1,0])
```

	Customer	City	Revenue
4	Karan	Mumbai	20000
3	Sneha	Bangalore	7000
2	Rahul	Delhi	15000
1	Priya	Mumbai	8000
0	Aman	Delhi	12000

✓ Demo Case Study

Sort products by revenue in descending order.

```
df.sort_values(by="Revenue", ascending=False)
```

	Customer	City	Revenue
4	Karan	Mumbai	20000

2	Rahul	Delhi	15000
1	Aman	Delhi	12000

1	Priya	Mumbai	8000
---	-------	--------	------

Use the following dataset for all questions.

3	Sneha	Bangalore	7000
---	-------	-----------	------

```
import pandas as pd

data = {
    "Customer": ["Aman", "Priya", "Rahul", "Sneha", "Karan"],
    "City": ["Delhi", "Mumbai", "Delhi", "Bangalore", "Mumbai"],
    "Revenue": [12000, 8000, 15000, 7000, 20000]
}

df = pd.DataFrame(data)
```

1

Display the entire dataset.

2

Select only the Revenue column.

3

Select the Customer and Revenue columns together.

4

Using loc, display the second row of the dataset.

5

Using iloc, display the third row of the dataset.

6

Using loc, retrieve the Revenue value of Rahul.

7

Filter the dataset to show customers whose Revenue is greater than 10000.

8

Display customers who belong to Mumbai.

9

Sort the dataset by Revenue in ascending order.